

Skapa egna namespaces

Man kan förstås skapa egna namespaces för att gruppera kod om man behöver. Detta blir vanligtvis dock aktuellt först då man har stora projekt, eller då man har en namnkonflikt orsakad av bibliotek man inte kan påverka. Moderna C++-bibliotek torde börja introducera namespaces för att undvika denna typ av problem, men alla har det ännu inte.

För att specificera att kod skall tillhöra ett visst namespace kan man göra på följande sätt:

```
namespace namespacesnamn {  
    deklarationer och definitioner  
}
```

Det enda som behövs är att säga var namespaceset börjar, vad det heter, samt var det slutar. All kod som skall bli innanför namespaceset kan lämnas orörd. Vi kan nu skapa ett eget namespace för att placera vår klass för matematiska vektorer. Vi kallar namespaceset för `math`. Man kan namnge namespaces enligt samma regler som gäller för variabler, funktioner, klasser m.m. Se [avsnittet Giltiga variabelnamn i Kapitel 2](#).

```
#include <vector>  
  
// definiera ett nytt namespace  
namespace math {  
  
    class vector {  
    public:  
        vector () {};  
    };  
}  
  
int main () {  
    // skapa en vektor av vektorer  
    std::vector<math::vector> VektorContainer;  
}
```

Vår egna klass `vector` tillhör nu namespaceset `math`, medan STL-klassen `vector` tillhör som normalt `std`. Vi kan genom att explicit ange vilken implementation av `vector` vi avser skapa en `vector` (container) av `vector` (matematiska vektorer).

Vi har i vårt exempel ovan endast en enda klass i vårt namespace, men vi kan inkludera i princip hur många definitioner som helst i ett och samma namespace. Vi kan lägga till en hypotetisk klass `matrix` på följande sätt:

```
#include <iostream>  
  
// definiera ett nytt namespace  
namespace math {  
  
    // en matematisk vektor  
    class vector {  
    public:  
        vector () {};  
    };  
  
    // en matematisk matris  
    class matrix {  
    public:  
        matrix () {};  
    };  
}
```

```
};

// en matematisk matrix
class matrix {
public:
    matrix (int X, int Y) : m_X(X), m_Y(Y) {};
private:
    int m_X, m_Y;
};

int main () {
    // skapa en vektor av vektorer
    std::vector<math::vector> VektorContainer;

    // skapa en matrix
    math::matrix M ( 3, 3 );
}
```

Vi kan även placera `typedef`-definitioner, enumereringar, vanliga funktioner m.m. i namespaces.

Namespaces och prototyper

I de fall vi tittat på ovan har all kod skrivits direkt in i namespacet. Detta fungerar bra för relativt små klasser, men om man vill presentera gränssnittet för t.ex. en stor klass och placera koden separat har vi två alternativ. Vi kan antingen *addera kod till namespacet* eller explicit ange till vilket namespace (och klass) en funktion/metod hör. Vi skall se på ett exempel där vi har tre funktioner som vi vill presentera i en headerfil:

```
#ifndef NAMESPACE5_H
#define NAMESPACE5_H

#include <string>

namespace StringManip {
    // definiera prototyper
    std::string toString (int Tal);
    int         toInt     (std::string Text);
    float        tofloat  (std::string Text);
}

#endif
```

Vi har nu definierat prototyper för tre funktioner, samt deklarerat att de tillhör namespacet `StringManip`. Vi har dock inte ännu visat koden för dessa, utan den finns i en separat fil. Hur kan vi då där specificera att vi implementerar funktionerna `toString()`, `toInt()` och `toFloat()` för namespacet `StringManip`? Om vi inte gör det kommer kompilatorn inte att kunna koppla samman prototyperna med den egentliga implementationen, och vi får ett länkningsfel. Det lättaste sättet är att kapsla in implementationen i en likadan namespace-deklaration som prototyperna. Vi kan då få någonting i stil med:

```
#include "Namespace5.h"

namespace StringManip {

    std::string toString (int Tal) {
        // koden bortlämnad
    }
}
```

```
int toInt (std::string Text) {  
    // koden bortlämnad  
}  
  
float tofloat (std::string Text) {  
    // koden bortlämnad  
}  
}
```

Funktionernas kod är i detta sammanhang irrelevant och bortlämnad. Detta exempel visar att namespaces är *öppna*, d.v.s. man kan lägga till kod till ett existerande namespace genom att helt enkelt kapsla in koden innanför ett namespace med samma namn. Notera att vi måste se till att vi exakt matchar våra implementationer mot prototyperna, precis som normalt, annars får vi länkningsproblem. Då man använder denna "öppna" syntax är det mycket enkelt att sprida ut koden som tillhör ett visst namespace över många filer utan problem.

Den andra metoden att lägga till kod till ett namespace innebär att varje symbol ges ett prefix som är namnet på namespace som symbolen skall placeras i. Vi kan då skriva funktionerna ovan på följande sätt:

```
#include "Namespace5.h"  
  
std::string StringManip::toString (int Tal) {  
    // koden bortlämnad  
}  
  
int StringManip::toInt (std::string Text) {  
    // koden bortlämnad  
}  
  
float StringManip::toFloat (std::string Text) {  
    // koden bortlämnad  
}
```

Varje funktion har nu prefixet `StringManip::`, som anger att funktionen i fråga tillhör namespace `StringManip`. Notera att det är samma syntax som används för att specificera vilken klass en metod tillhör. Här kan många likheter märkas, eftersom en klass även kan ses som en form av namespace, och dess metoder och definitioner är kapslade "under samma tak". Vad händer då om vi har en klass som tillhör ett namespace och vars metoder vi vill implementera separat, kanske i en skild fil? En enkel lösning är ju att skriva metoderna som normalt och sedan kapsla in dem innanför ett enda namespace, såsom vi gjorde i den första versionen tidigare i detta avsnitt. En annan lösning är att använda ett prefix, precis som vi gjorde ovan. Vi kan ta som exempel en klass `Coordinate` som vi lägger till till namespace `math`:

```
#include <iostream>  
  
namespace math {  
    class Coordinate {  
    public:  
        // konstruktor  
        Coordinate (float X, float Y);  
  
        // accessera medlemmar  
        float x ();  
        float y ();  
        void setX (float X);  
        void setY (float Y);  
    };  
}
```

```
private:
    // medlemmar
    float m_X, m_Y;
};
}

// implementera alla metoder.
math::Coordinate::Coordinate (float X, float Y) { m_X = X; m_Y = Y; };
float math::Coordinate::x () { return m_X; };
float math::Coordinate::y () { return m_Y; };
void math::Coordinate::setX (float X) { m_X = X; };
void math::Coordinate::setY (float Y) { m_Y = Y; };

int main () {
    using namespace std;

    math::Coordinate C (1,1);
    std::cout << "Koordinater: (" << C.x () << ", "
                << C.y () << ")" << endl;
}
```

Vi har här skrivit metoderna helt normalt, med den enda skillnaden att de fått ett extra prefix `math::`. Implementationen av metoderna kan bli ganska svårläst då man använder denna metod, eftersom det blir ett extra prefix, men istället vet läsaren direkt vilket namespace (och förstås även vilken klass) metoden tillhör. Vilken metod man väljer för egen kod spelar ingen roll.

[Förutgående](#)
Använda namespaces

[Hem](#)
[Upp](#)

[Nästa](#)
Alias